Advanced I/O

Section: 5 — File System Operations and Streams

Subtopic: 03 — Advanced I/O

Estimated Duration: 1.5–2 hours

Theory Practical

Topics covered: Watching files with `fs.watch`, file system error handling, common error codes (ENOENT, EACCES), graceful error recovery

Learning Objectives — This Subtopic

- Use `fs.watch()` to monitor files and directories for changes in real time
- Understand the event types emitted by the file watcher and respond to them appropriately
- Identify common file system error codes and their meaning
- Implement robust error handling patterns for `fs` operations to prevent application crashes

1. Watching Files with `fs.watch()`

Many development tools — live-reload servers, build systems, test runners — need to detect when files change on disk. Node.js provides `fs.watch()` for this purpose. It uses operating system-level notifications (inotify on Linux, FSEvents on macOS, ReadDirectoryChangesW on Windows) to detect changes efficiently without polling.

Create a project folder for this section:

```
mkdir advanced-io && cd advanced-io
```

1.1 Watching a Single File

`watch-file.js`

```
const fs = require('fs');

// Create a file to watch
fs.writeFileSync('watched.txt', 'Initial content\n');
console.log('Watching watched.txt for changes...');
console.log('(Modify the file in another terminal, then press Ctrl+C to stop)\n');

const watcher = fs.watch('watched.txt', (eventType, filename) => {
    console.log(`Event: ${eventType} | File: ${filename}`);
});

watcher.on('error', (err) => {
    console.error('Watcher error:', err.message);
});
```

`node watch-file.js`

In a **separate terminal window**, modify the file:

macOS/Linux: `echo "New line added" >> watched.txt`

Windows: `echo New line added >> watched.txt`

> **Output:**
>
> ```
> Watching watched.txt for changes...
> (Modify the file in another terminal, then press Ctrl+C to stop)
>
> Event: change | File: watched.txt
> ```

What is happening? `fs.watch()` takes a file path and a callback. The callback receives two arguments: `eventType` (either `'rename'` or `'change'`) and `filename` (the name of the file that triggered the event). The `'change'` event fires when a file's contents are modified. The `'rename'` event fires when a file is created, deleted, or renamed.

1.2 Watching a Directory

`fs.watch()` can also monitor an entire directory. On supported platforms, you can enable `recursive: true` to watch subdirectories as well.

watch-directory.js

```
const fs = require('fs');

// Create a test directory structure
const testDir = 'watch-test';
if (!fs.existsSync(testDir)) {
    fs.mkdirSync(testDir);
}

console.log(`Watching directory: ${testDir}/`);
console.log('Try creating, modifying, or deleting files inside it.\n');

const watcher = fs.watch(testDir, { recursive: true }, (eventType, filename) => {
    const timestamp = new Date().toLocaleTimeString('en-GB');
    console.log(`[${timestamp}] ${eventType}: ${filename}`);
});

watcher.on('error', (err) => {
    console.error('Watcher error:', err.message);
});

// Clean up on exit
process.on('SIGINT', () => {
    console.log('\nStopping watcher...');
    watcher.close();
    process.exit(0);
});
```

```
node watch-directory.js
```

In a separate terminal, create and modify files inside the `watch-test` directory:

```
echo "hello" > watch-test/test1.txt
```

```
echo "update" >> watch-test/test1.txt
```

macOS/Linux: `rm watch-test/test1.txt` | Windows: `del watch-test\test1.txt`

Output:

```
Watching directory: watch-test/
Try creating, modifying, or deleting files inside it.

[14:32:07] rename: test1.txt
[14:32:07] change: test1.txt
[14:32:15] change: test1.txt
[14:32:22] rename: test1.txt
```

What is happening? When a file is first created, it triggers a `'rename'` event (a new name appeared in the directory), often followed immediately by a `'change'` event (the file contents were written). Modifying the file triggers `'change'`. Deleting it triggers `'rename'` (the name disappeared from the directory).

> **Important:** The `recursive: true` option is supported on macOS and Windows, but **not on Linux**. On Linux, `fs.watch()` only monitors the immediate directory. If you need recursive watching across all platforms, see the note on chokidar at the end of this section.

1.3 Debouncing Watch Events

A common issue with `fs.watch()` is that a single file save can trigger multiple events in rapid succession. Text editors often write to a temporary file and then rename it, or perform multiple write operations. This results in duplicate event callbacks.

The solution is to **debounce** the events — wait a short period after receiving an event before acting on it, and ignore any additional events during that window:

`watch-debounced.js`

```
const fs = require('fs');

// Create a file to watch
fs.writeFileSync('config.json', JSON.stringify({ port: 3000 }, null, 2));

const debounceTimers = {};
const DEBOUNCE_MS = 100;

const watcher = fs.watch('config.json', (eventType, filename) => {
    // If a timer already exists for this file, clear it
    if (debounceTimers[filename]) {
        clearTimeout(debounceTimers[filename]);
```

```
    }

    // Set a new timer — only act after events have settled
    debounceTimers[filename] = setTimeout(() => {
        console.log(`[${eventType}] ${filename} — processing change`);

        // Read and display the updated content
        try {
            const content = fs.readFileSync(filename, 'utf8');
            console.log('Current content:', content);
        } catch (err) {
            console.log('File may have been deleted:', err.code);
        }

        delete debounceTimers[filename];
    }, DEBOUNCE_MS);
});

console.log('Watching config.json (debounced)...');
console.log('Modify the file to see a single event per save.\n');

watcher.on('error', (err) => {
    console.error('Watcher error:', err.message);
});
```

`node watch-debounced.js`

Modify `config.json` in another terminal or a text editor:

Output:

```
Watching config.json (debounced)...
Modify the file to see a single event per save.

[change] config.json — processing change
Current content: {
  "port": 8080
}
```

What is happening? Without debouncing, you might see 2–3 `'change'` events for a single save. The debounce pattern uses `setTimeout()` to wait 100ms after the last event before executing the handler. If another event fires within that window, the previous timer is cancelled and a new one starts. This ensures you process each save exactly once.

1.4 A Note on `chokidar`

`fs.watch()` is sufficient for straightforward file watching, but it has known limitations: inconsistent behaviour across operating systems, no recursive support on Linux, and occasional duplicate or missing events depending on the editor and OS.

For production file-watching tools, the community-standard solution is the `chokidar` package. It wraps the native file system events with normalisation, reliable recursive watching on all platforms, and built-in debouncing. You already know how to install third-party packages from Section 2:

```
npm install chokidar
```

We will not cover chokidar's API in this section, but know that it exists and is the foundation of tools like `nodemon` and webpack's watch mode. If you build a tool that depends on reliable file watching in production, use chokidar instead of raw `fs.watch()`.

---

2. File System Error Handling

Every `fs` operation can fail. A file might not exist, the process might lack permissions, or the disk might be full. Node.js represents these failures through error objects with a `code` property — a short string that identifies the type of failure.

2.1 Common Error Codes

Error Code	Meaning	Typical Cause
`ENOENT`	No such file or directory	The path does not exist
`EACCES`	Permission denied	The process lacks read/write permission
`EISDIR`	Is a directory	Attempted to read/write a directory as a file
`EEXIST`	File already exists	

		Attempted to create a file/ directory that already exists (with exclusive flag)
`EMFILE`	Too many open files	The process has exceeded the OS limit for open file descriptors
`ENOSPC`	No space left on device	The disk is full

2.2 Handling Errors in Async Operations

In Section 5-1, you used the callback form of `fs` methods. The error-first callback pattern makes error handling straightforward — the first argument is always the error (or `null` if the operation succeeded).

`error-callback.js`

```
const fs = require('fs');

// Attempt to read a file that does not exist
fs.readFile('does-not-exist.txt', 'utf8', (err, data) => {
    if (err) {
        console.log('Error object properties:');
        console.log('  code:    ', err.code);
        console.log('  message: ', err.message);
        console.log('  syscall: ', err.syscall);
        console.log('  path:    ', err.path);
        return;
    }

    console.log('File contents:', data);
});
```

`node error-callback.js`

Output:

```
Error object properties:
  code:     ENOENT
  message:  ENOENT: no such file or directory, open 'does-not-exist.txt'
  syscall:  open
  path:     does-not-exist.txt
```

What is happening? The error object from `fs` operations includes several useful properties: `code` identifies the error type programmatically, `message` gives a human-readable description, `syscall` tells you which system call failed, and `path` shows the file path involved. The `code` property is what you should use in conditional logic — never parse the `message` string.

2.3 Handling Errors with Promises (async/await)

The `fs/promises` API (which you saw briefly in Section 5-1) throws errors that you catch with `try/catch`. The error objects have the same properties:

error-async.js

```
const fs = require('fs/promises');

async function readConfig(filePath) {
    try {
        const data = await fs.readFile(filePath, 'utf8');
        console.log('Config loaded:', data);
    } catch (err) {
        if (err.code === 'ENOENT') {
            console.log(`File not found: ${filePath}`);
            console.log('Creating default config...');

            const defaultConfig = JSON.stringify({ port: 3000, debug: false }, null, 2);
            await fs.writeFile(filePath, defaultConfig);
            console.log('Default config created.');
        } else if (err.code === 'EACCES') {
            console.error(`Permission denied: ${filePath}`);
            console.error('Check file permissions and try again.');
        } else {
            // Re-throw unexpected errors
            throw err;
        }
    }
```

```
}

readConfig('app-config.json');
```

```
node error-async.js
```

> **Output:**
>
> ```
> File not found: app-config.json
> Creating default config...
> Default config created.
> ```

Run it again — the file now exists:

```
node error-async.js
```

> **Output:**
>
> ```
> Config loaded: {
>   "port": 3000,
>   "debug": false
> }
> ```

What is happening? This is a practical pattern: attempt to load a configuration file, and if it does not exist (`ENOENT`), create one with sensible defaults instead of crashing. The `code` property lets you handle each error type differently — a missing file is recoverable, but a permission error might require the user to take action.

> **Key Insight:** Always handle expected error codes explicitly and re-throw unexpected ones. Swallowing all errors silently with a generic `catch` makes debugging extremely difficult. If you do not know how to handle a particular error, let it propagate — a crash with a clear error message is better than silent corruption.

2.4 Handling Errors in Sync Operations

Synchronous `fs` methods throw errors directly. Wrap them in `try/catch` :

`error-sync.js`

```
const fs = require('fs');

function fileExists(filePath) {
    try {
        fs.accessSync(filePath, fs.constants.R_OK);
        return true;
    } catch (err) {
        return false;
    }
}

// Test with an existing file and a non-existing one
fs.writeFileSync('real-file.txt', 'I exist');

console.log('real-file.txt exists?', fileExists('real-file.txt'));
console.log('fake-file.txt exists?', fileExists('fake-file.txt'));

// Attempting to read a directory as a file
try {
    fs.readFileSync('.', 'utf8');
} catch (err) {
    console.log(`\nError reading directory as file:`);
    console.log(`  Code: ${err.code}`);
    console.log(`  Message: ${err.message}`);
}
```

`node error-sync.js`

Output:

```
real-file.txt exists? true
fake-file.txt exists? false

Error reading directory as file:
  Code: EISDIR
  Message: EISDIR: illegal operation on a directory, read
```

`fs.accessSync()` checks whether the process has the specified access to a file. `fs.constants.R_OK` checks for read permission. If the file does not exist or is not readable, it throws an error. This is a clean way to check file existence without reading the full contents.

> **Warning:** Avoid using `fs.existsSync()` followed by `fs.readFileSync()` as a two-step check-then-act pattern. Between the check and the read, another process could delete the file. This is called a **TOCTOU** (time-of-check-to-time-of-use) race condition. Instead, attempt the operation directly and handle the error if it fails.

2.5 Putting It Together — A Resilient File Reader

This example combines several error handling patterns into a function that handles multiple failure modes gracefully:

resilient-reader.js

```
const fs = require('fs/promises');
const path = require('path');

async function safeReadJSON(filePath) {
    const resolvedPath = path.resolve(filePath);

    try {
        const raw = await fs.readFile(resolvedPath, 'utf8');
        const parsed = JSON.parse(raw);
        return { success: true, data: parsed };
    } catch (err) {
        switch (err.code) {
            case 'ENOENT':
                return { success: false, error: `File not found: ${resolvedPath}` };

            case 'EACCES':
                return { success: false, error: `Permission denied: ${resolvedPath}` };

            case 'EISDIR':
                return { success: false, error: `Path is a directory, not a file: $
{resolvedPath}` };

            default:
                // SyntaxError from JSON.parse has no 'code' property
                if (err instanceof SyntaxError) {
                    return { success: false, error: `Invalid JSON in ${resolvedPath}: $
{err.message}` };
                }

                // Unexpected error — re-throw
                throw err;
```

```
        }
    }
}

// Test all scenarios
async function runTests() {
    // 1. Valid JSON file
    await fs.writeFile('valid.json', '{"name": "test", "version": 1}');
    console.log('Test 1 — Valid file:');
    console.log(await safeReadJSON('valid.json'));

    // 2. Non-existent file
    console.log('\nTest 2 — Missing file:');
    console.log(await safeReadJSON('missing.json'));

    // 3. Invalid JSON
    await fs.writeFile('broken.json', '{ name: invalid }');
    console.log('\nTest 3 — Invalid JSON:');
    console.log(await safeReadJSON('broken.json'));

    // 4. Directory instead of file
    console.log('\nTest 4 — Directory path:');
    console.log(await safeReadJSON('.'));
}

runTests();
```

```
node resilient-reader.js
```

Output:

```
Test 1 — Valid file:
{ success: true, data: { name: 'test', version: 1 } }

Test 2 — Missing file:
{ success: false, error: 'File not found: /Users/you/advanced-io/missing.json' }

Test 3 — Invalid JSON:
{
  success: false,
  error: "Invalid JSON in /Users/you/advanced-io/broken.json: Expected property name or '}' in JSON at position
2"
}

Test 4 — Directory path:
{ success: false, error: 'Path is a directory, not a file: /Users/you/advanced-io' }
```

What is happening? The function returns a consistent result shape (`{ success, data/error }`) regardless of what went wrong. It distinguishes between file system errors (using `err.code`) and parsing errors (using `instanceof SyntaxError`). The caller gets a clear, actionable message for each failure type without the function crashing.

Summary

Concept	Key Point
`fs.watch()`	Monitors files or directories for changes using OS-level notifications; callback receives `eventType` and `filename`
Event types	`'change'` for content modifications, `'rename'` for creation, deletion, or renaming
Debouncing	Use `setTimeout()` to consolidate rapid duplicate events into a single handler execution
`recursive` option	Watches subdirectories on macOS/Windows; not supported on Linux (use chokidar for cross-platform)
`ENOENT`	File or directory does not exist — often recoverable by creating defaults
`EACCES`	Permission denied — requires the user or process to fix file permissions
`EISDIR`	Attempted file operation on a directory path
Error handling pattern	Switch on `err.code` for known errors, re-throw unknown ones; never swallow errors silently
TOCTOU	Avoid check-then-act patterns; attempt the operation and handle the error instead

Interview Questions

1. How does `fs.watch()` detect file changes, and what are its limitations?
Expected answer: `fs.watch()` uses OS-level file system notifications (inotify on Linux, FSEvents on macOS, ReadDirectoryChangesW on Windows). Its limitations include: inconsistent behaviour across operating systems, no recursive watching on Linux, possible duplicate events for a single file save, and the `filename` argument may be `null` on some platforms.

2. Why might a single file save trigger multiple `fs.watch()` events, and how do you handle it?
Expected answer: Text editors often perform multiple operations for a single save — writing to a temp file, renaming, or writing in multiple steps. This triggers several events. The solution is to debounce the events using `setTimeout()`: reset a timer on each event, and only execute the handler once events have stopped arriving for a set interval (e.g., 100ms).

3. What is `ENOENT` and how should you handle it?
Expected answer: `ENOENT` stands for "Error NO ENTry" — the file or directory does not exist. Handling depends on context: you might create a default file, prompt the user, or return a meaningful error message. The key is to check `err.code === 'ENOENT'` rather than parsing the error message string.

4. What is a TOCTOU race condition in file system operations?
Expected answer: TOCTOU (time-of-check-to-time-of-use) occurs when you check whether a file exists and then act on it — but between the check and the action, the file state changes (e.g., another process deletes it). The correct approach is to attempt the operation directly and handle the error if it fails, rather than checking first.

5. When would you use `chokidar` instead of `fs.watch()`?
Expected answer: When you need reliable, cross-platform file watching — especially recursive directory watching on Linux (which `fs.watch()` does not support). Chokidar normalises the OS-level differences and provides a consistent API with built-in debouncing and event filtering. Production tools like nodemon and webpack use chokidar internally.

6. In the error-first callback pattern, why is it important to `return` after handling the error?
Expected answer: Without a `return` (or an `else` block), the code after the error check continues to execute, attempting to use the `data` parameter that is `undefined` when an error occurred. This leads to confusing secondary errors that mask the original problem.

Practical Assignment — Directory Change Logger

Objective: Build a command-line tool that watches a directory and logs all file system changes to a structured log file, with debouncing and proper error handling.

Step 1: Create a project folder and set up the workspace:

```
mkdir dir-watcher && cd dir-watcher
```

```
mkdir watched-folder
```

Step 2: Create the watcher tool:

watcher.js

```
const fs = require('fs');
const path = require('path');

// --- Configuration ---
const WATCH_DIR = path.join(__dirname, 'watched-folder');
const LOG_FILE = path.join(__dirname, 'changes.log');
const DEBOUNCE_MS = 150;

// --- Validate target directory ---
try {
    fs.accessSync(WATCH_DIR, fs.constants.R_OK);
} catch (err) {
    if (err.code === 'ENOENT') {
        console.error(`Error: Directory does not exist: ${WATCH_DIR}`);
        console.error('Create it first: mkdir watched-folder');
    } else if (err.code === 'EACCES') {
        console.error(`Error: No read permission for: ${WATCH_DIR}`);
```

```
    } else {
        console.error(`Unexpected error: ${err.message}`);
    }
    process.exit(1);
}

// --- Log writer ---
const logStream = fs.createWriteStream(LOG_FILE, { flags: 'a' });

function logChange(eventType, filename) {
    const timestamp = new Date().toISOString();
    const filePath = path.join(WATCH_DIR, filename);

    let status = 'unknown';
    try {
        fs.accessSync(filePath);
        status = eventType === 'rename' ? 'created' : 'modified';
    } catch (err) {
        if (err.code === 'ENOENT') {
            status = 'deleted';
        }
    }

    const entry = `[${timestamp}] ${status.toUpperCase().padEnd(8)} ${filename}`;
    logStream.write(entry + '\n');
    console.log(entry);
}

// --- Debounce map ---
const timers = {};

// --- Start watching ---
console.log(`Watching: ${WATCH_DIR}`);
console.log(`Logging to: ${LOG_FILE}`);
console.log('Press Ctrl+C to stop.\n');

const watcher = fs.watch(WATCH_DIR, (eventType, filename) => {
    if (!filename) return; // Some platforms may not provide filename

    if (timers[filename]) {
        clearTimeout(timers[filename]);
    }

    timers[filename] = setTimeout(() => {
        logChange(eventType, filename);
        delete timers[filename];
    }, DEBOUNCE_MS);
```

```
});

watcher.on('error', (err) => {
    console.error('Watcher error:', err.message);
    logStream.write(`[${new Date().toISOString()}] ERROR    ${err.message}\n`);
});

// --- Graceful shutdown ---
process.on('SIGINT', () => {
    console.log('\nShutting down watcher...');
    watcher.close();
    logStream.end(() => {
        console.log('Log file closed. Goodbye.');
        process.exit(0);
    });
});
```

Step 3: Run the watcher:

`node watcher.js`

Step 4: In a separate terminal, create and modify files inside `watched-folder`:

`echo "first file" > watched-folder/notes.txt`

`echo "second file" > watched-folder/data.csv`

`echo "update" >> watched-folder/notes.txt`

macOS/Linux: `rm watched-folder/data.csv` | Windows: `del watched-folder\data.csv`

Step 5: Stop the watcher with `Ctrl` + `C`, then check the log file:

`cat changes.log` (macOS/Linux) or `type changes.log` (Windows)

Expected output:

> Output:

```
[2025-01-15T14:32:07.123Z] CREATED  notes.txt
[2025-01-15T14:32:12.456Z] CREATED  data.csv
[2025-01-15T14:32:18.789Z] MODIFIED notes.txt
[2025-01-15T14:32:25.012Z] DELETED  data.csv
```

```
dir-watcher/
├── watcher.js
├── changes.log
└── watched-folder/
    └── notes.txt
```

Bonus Challenge

Extend the watcher to also log the file size for `CREATED` and `MODIFIED` events. Use `fs.statSync()` to get the size, and handle the case where the file might be deleted between the event firing and the stat call (TOCTOU). Format the log entry as: `[timestamp] MODIFIED notes.txt (1.2 KB)`.